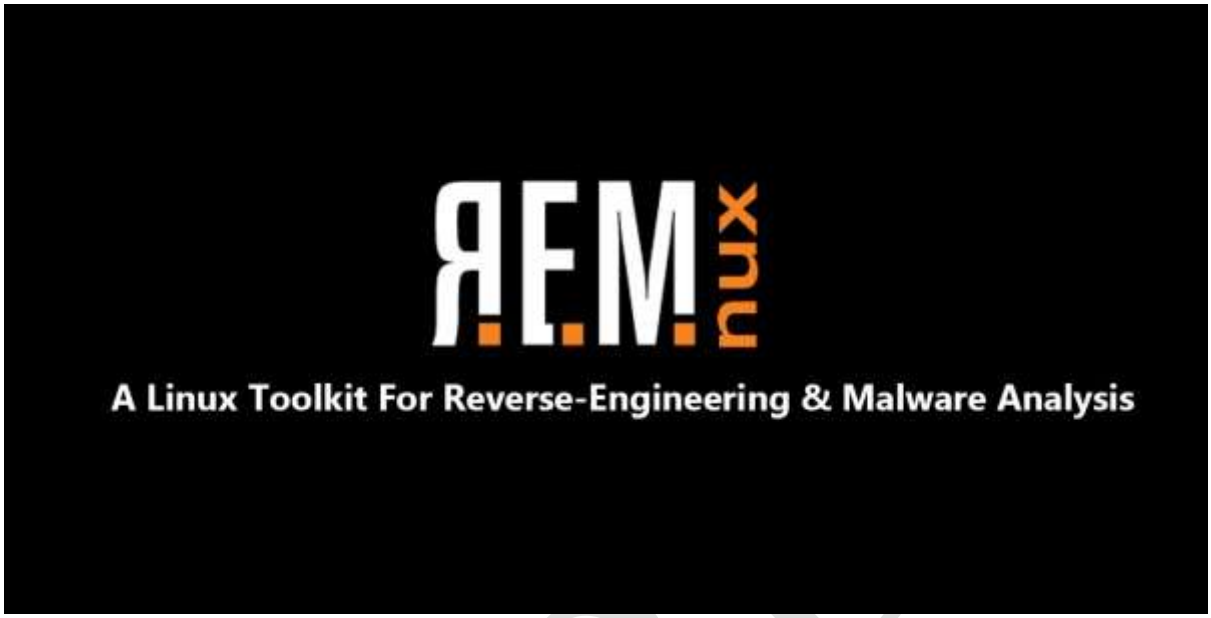
REMnux

Static/Dynamic Analysis and Automated Evidence Preprocessing (REMnux)



Malware analysis is a critical component of incident response, requiring specialized environments to safely dissect malicious software.
This module introduces **REMnux**, a Linux distribution pre-configured with a robust toolkit for static and dynamic analysis. We examine **oledump.py** for inspecting malicious macros in Microsoft Office documents and **INetSim** for simulating fake network services to observe command-and-control (C2) behaviors safely. Furthermore, the content details memory forensic workflows using **Volatility 3**, focusing on process identification, command-line extraction, and injected code detection.
By utilizing automated preprocessing scripts and string extraction utilities, security professionals can expedite the identification of Indicators of Compromise (IOCs) while maintaining environment integrity.

This **documentation** and **learning** approach is strongly inspired by the structured, hands-on content provided by **TryHackMe**, which significantly contributed to practical understanding of "REMnux Tool Basic" concepts.

DOCUMENTED BY
Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

1. Introduction to REMnux

REMnux is a specialized Linux distribution designed for the analysis of malicious programs and documents. It provides a "laboratory" environment containing industry-standard tools like Wireshark, YARA, and Volatility.

- **Core Purpose:** To provide a **sandbox-like** environment where analysts can examine malware without risking the infection of their primary host system.
- **Key Capabilities:** Static analysis of documents, network traffic simulation, memory forensics, and binary de-obfuscation.

2. Static File Analysis: oledump.py

Many threat actors utilize **OLE2** files (Structured Storage) such as Excel or Word documents to deliver malware via **VBA Macros**. **oledump.py** is the primary tool for analyzing these containers.

Command Section: oledump.py

Command	Description
oledump.py [file]	Lists the data streams inside the document.
-s [index]	**Selects** a specific data stream for inspection.
--vbadecompress	Decompresses VBA code into a human-readable format.

The "M" Indicator

In oledump results, streams marked with a capital **"M"** indicate that a **Macro** is present. Streams marked with a lowercase **"m"** contain code but may not be executable macros.

3. Dynamic Analysis: Fake Networking with INetSim

Malware often tries to connect to a server to download more payloads. **INetSim (Internet Services Simulation Suite)** allows analysts to trick the malware into thinking it has an active internet connection.

Configuration and Execution

1. **Configure:** Edit /etc/inetsim/inetsim.conf to set the dns_default_ip to the REMnux machine's IP. This ensures the malware's DNS requests resolve to your lab machine.
2. **Start:** Run sudo inetsim.
3. **Simulation:** The tool spawns fake HTTP, HTTPS, SMTP, and FTP servers.

Connection Reports

After stopping INetSim, a report is generated in /var/log/inetsim/report/. This file logs every URL, protocol, and HTTP method (e.g., **GET**, **POST**) the malware attempted to use.

WEIRDO

4. Memory Investigation: Volatility 3

Memory forensics involves analyzing the **RAM** captured from an infected system to find volatile artifacts like running processes and network connections.

Volatility 3 Plugin Reference

Plugin	Purpose
pstree	Displays processes in a parent-child relationship (Tree view).
pslist	Lists all currently active processes.
cmdline	Shows the specific arguments used to launch a process.
malfind	Identifies memory regions containing potentially **injected code**.
filescan	Scans for file objects residing in memory.
dlllist	Lists the Dynamic Link Libraries (DLLs) loaded by processes.

Automated Preprocessing

Professionals often use a **Loop Statement** to run multiple plugins at once and save the results to text files for faster searching:

```
for plugin in windows.pslist windows.pstree; do vol3 -f wcry.mem $plugin >
output.$plugin.txt; done
```

WEIRDO

5. Practical Analysis Scenarios

Malicious Excel Analysis (Task 3)

- **Work Done: I practiced** analyzing the file agenttesla.xlsm. **I researched** the data streams and **found** a macro in stream **A4** (marked with 'M').
- **De-obfuscation: I practiced** using --vbadecompress to reveal a PowerShell script hidden with * and ^ characters. **I solved** the de-obfuscation by using **CyberChef's Find/Replace** tool.
- **Term Explanation:** Invoke-WebRequest was the command **I founded** in the script, which is used to download external files.
- **Findings: I founded** the script was attempting to download Doc-3737122pdf.exe and store it in a variable named $TempFile.
- **Additional Work: I researched** possible_malicious.docx and **found** it contained **15 data streams**, with the macro located at stream **8**.

Simulated Network Investigation (Task 4)

- **Work Done: I practiced** setting up the INetSim fake network. **I solved** the task by using wget to request a file from the fake server.
- **Flag Retrieval: I founded** the flag THM{FAKE_NETWORK_FLAG} by downloading flag.txt from the simulated network.
- **Report Analysis:** After stopping the service, **I researched** the generated report and **found** that the **GET** method was used to retrieve the flag file.

Memory Evidence Preprocessing (Task 5)

- **Work Done: I practiced** analyzing the wcry.mem image (WannaCry ransomware) using Volatility.
- **Injected Code Identification: I practiced** with the malfind plugin. **I founded** that the first suspected process was tasksche.exe (PID 1940), and the second was @WanaDecryptor@ (PID 740).
- **Process Analysis: I practiced** using pstree to view the process hierarchy and DllList to find the path of the malicious binary. **I founded** it located at C:\Intel\ivecuqmanpnirkt615\@WanaDecryptor@.exe.
- **String Extraction: I solved** the string search by using the Linux strings utility with -e l to find **little-endian** unicode strings within the memory image.

WEIRDO

Conclusion and Cheatsheet

Conclusion

Mastering the REMnux toolkit allows for an efficient, standardized approach to malware analysis. By combining static inspection of document streams using **oledump.py** with dynamic behavioral observation via **INetSim**, analysts can uncover the full execution chain of a threat. Memory forensics with **Volatility 3** provides the final piece of the puzzle, revealing how malware lives in the system's RAM and uncovering hidden or injected code. Automated preprocessing remains the industry best practice for handling large-scale evidence efficiently.

Fast Revision Cheatsheet

- **oledump.py -s [X]:** Selects stream X for analysis.
- **M Indicator:** Means a macro is present in an OLE file.
- **INetSim Default Port:** 80/443 for fake web traffic.
- **dns_default_ip:** Critical setting in inetsim.conf to redirect traffic.
- **vol3 -f [file] windows.pslist:** Basic command to list memory processes.
- **pstree:** Best plugin for finding "orphaned" malicious processes.
- **strings -e l:** Extracts **Unicode (16-bit little-endian)** strings.
- **Invoke-WebRequest:** PowerShell cmdlet used for file downloads.